Aalto University
School of Arts, Design
and Architecture

Check point no.8
Topic: Distributed Mixture-of-Experts (MoE) systems

Phuong Ngo Pitkänen and Amirreza Jafariandehkordi

Load imbalance in distributed MoE training wastes GPU compute at scale

Problem

In Mixture-of-Experts (MoE)models, a learned router sends each token to a small subset of N expert sub-networks. Routing is non-uniform — the router develops preferences for certain experts, which receive disproportionately more tokens.In distributed training, all ranks synchronise at each step boundary, so ***the slowest GPU determines step latency for the entire cluster***.

What we observe

At 2–4 GPUs (Phase 1), the workload is near-balanced and wasted fraction stays below 2%.

At 8 GPUs with a heavy expert distribution (Phase 2), wasted fraction rises to 5–9% and persists throughout training.

Intervention

A threshold-triggered rebalancing planner monitors an imbalance metric at every training step. When the metric exceeds threshold θ, the planner proposes ***a new expert-to-GPU assignment via greedy load-sorted mapping and estimates the predicted latency savings.***

What the parameter study set out to answer

5,400 planner configurations evaluated via offline simulation on LUMI training logs (2–16 GPUs)

1

Which signals to use?

Which imbalance signals respond to threshold tuning and which do not?

2

How much can we save?

How world size and workload severity determine recoverable latency per trigger?

3

When to check and wait?

The effect of evaluation interval and cooldown on per-trigger savings quality

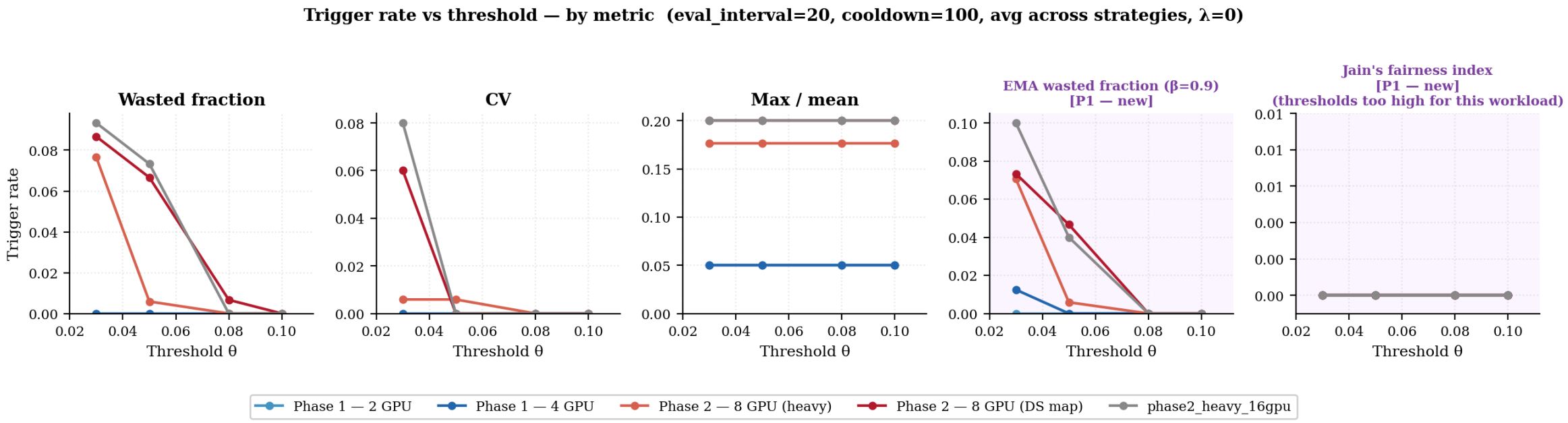
4

What is the best trade-off?

The Pareto frontier across all 5,400 configurations: trigger frequency vs savings per trigger

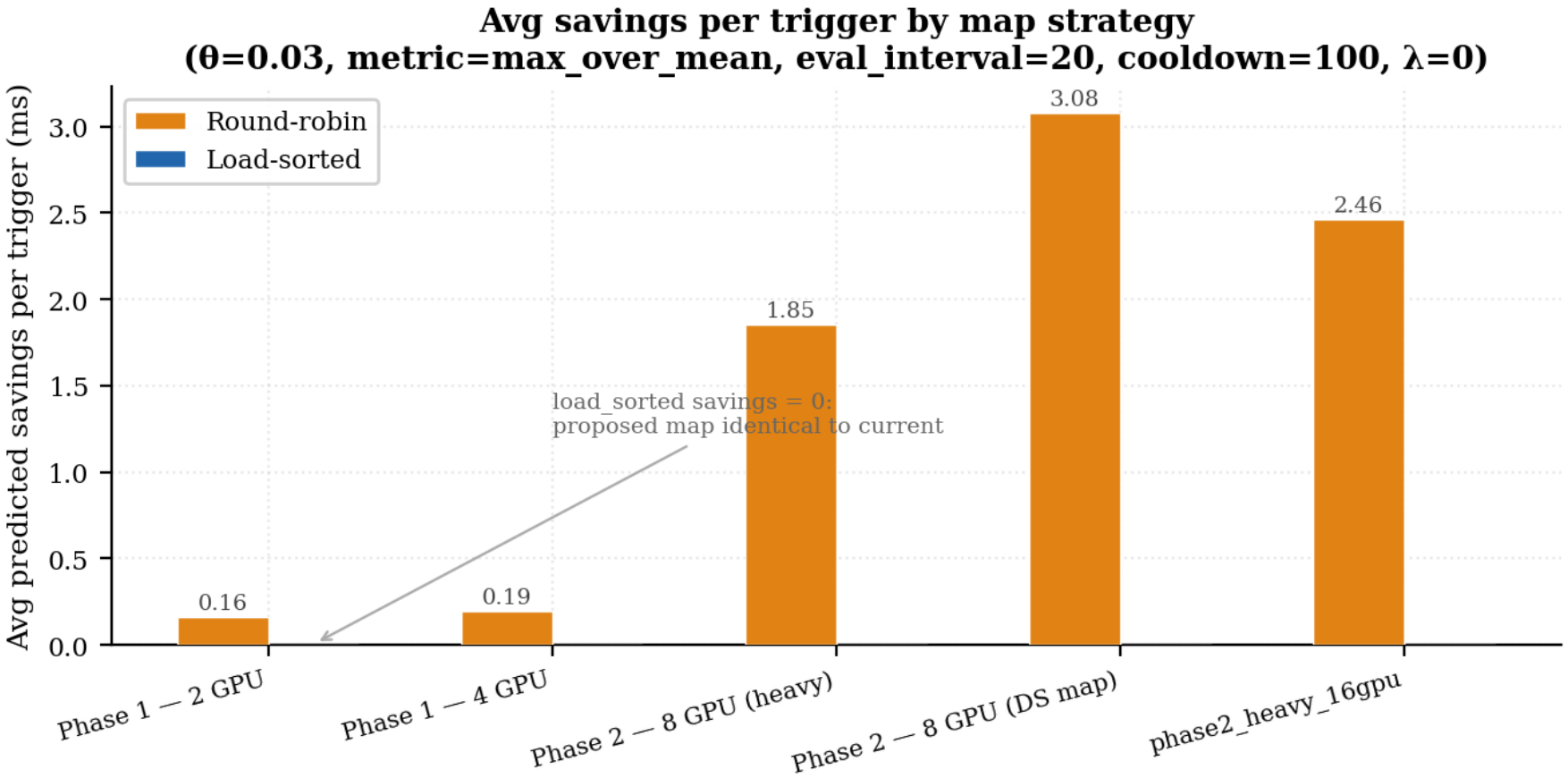
Finding 1: Not all metrics can be tuned

Question: as we raise the detection threshold, does the metric fire less often? The only metrics worth using are wasted fraction, CV, and EMA.



Finding 2: Initial expert-to-GPU mapping quality determines the upper bound of per-trigger savings throughout training.

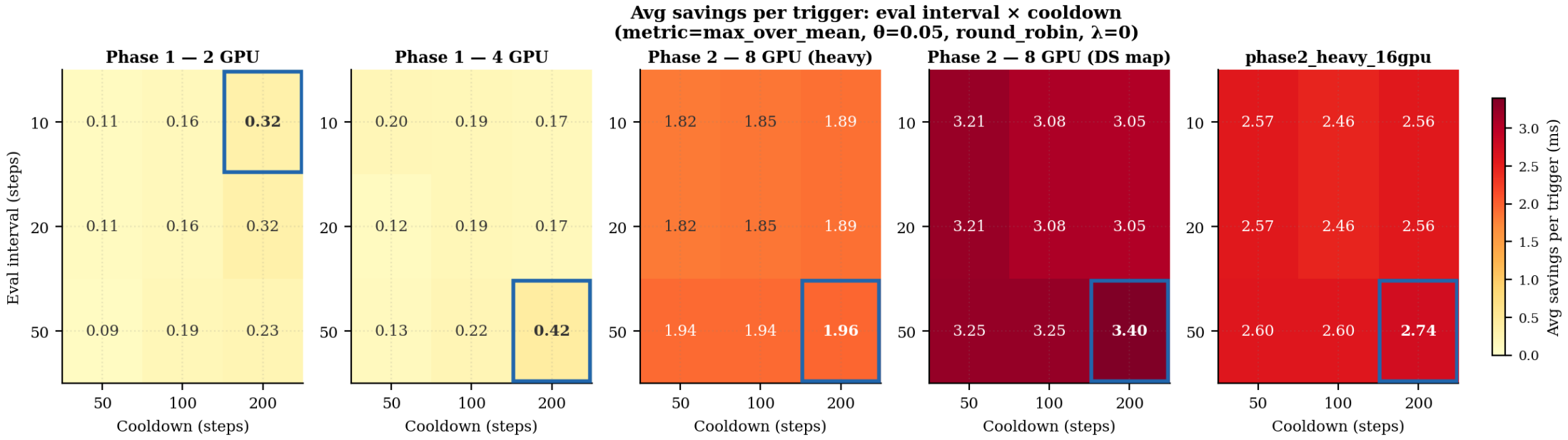
Question: if you start with round-robin and switch to load-sorted, how many milliseconds of compute waste do you save on average per rebalancing trigger.



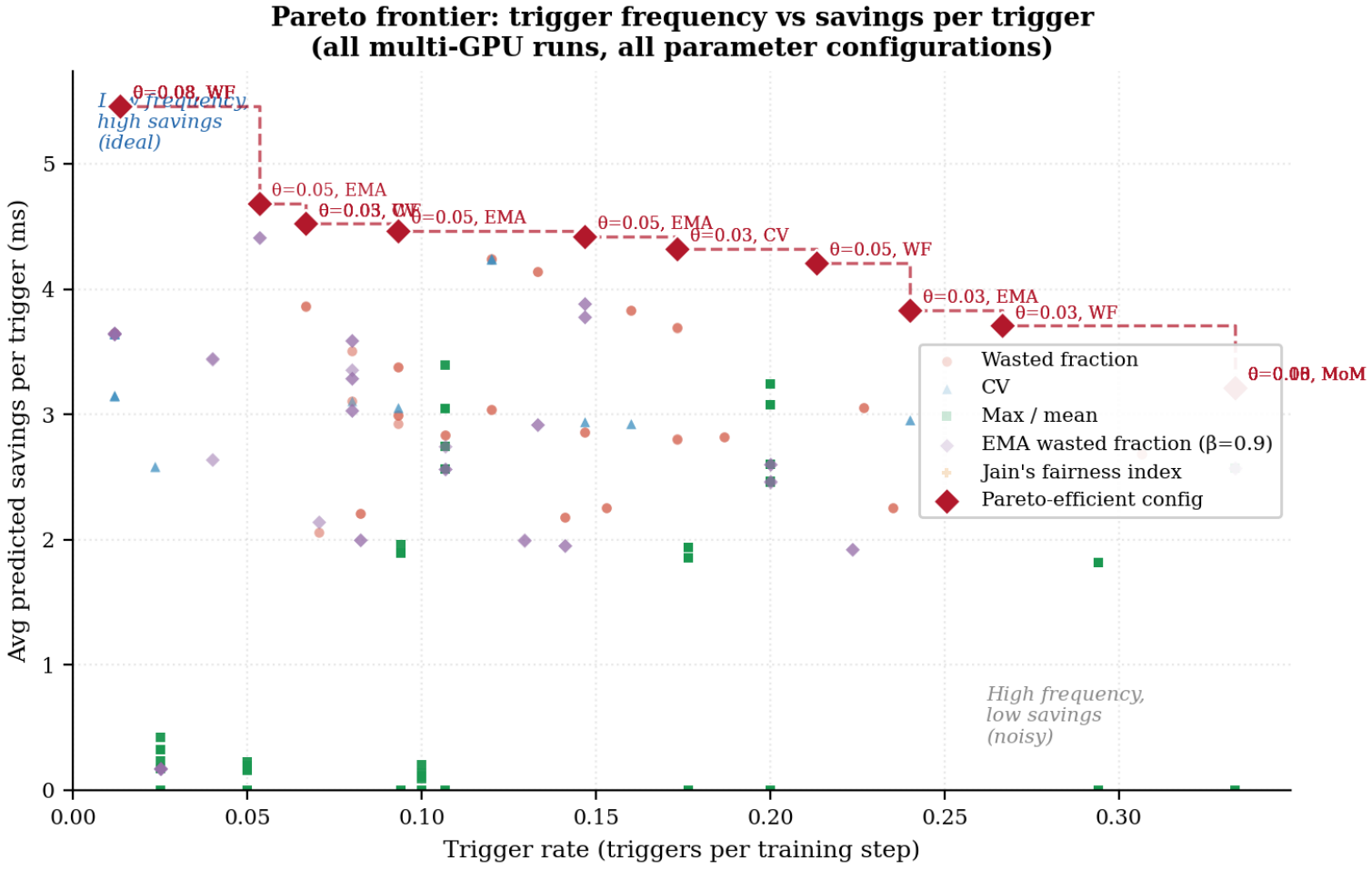
A!

Finding 3: Longer cooldown selectively suppresses low-value triggers while preserving high-value ones

Moving from cooldown 50 to cooldown 200 consistently increases average savings per trigger across all runs. When the cooldown is long, only genuinely sustained and severe imbalance events survive the wait.



Finding 4: The best overall configuration is θ=0.05 with EMA wasted fraction



- EMA wasted fraction holds three of the ten Pareto-efficient configurations because smoothing filters out single-step noise and only triggers when imbalance has beenbuilding consistently over several steps, making each trigger more meaningful.
- Raw wasted fraction also reaches the frontier but only at a higher threshold of θ=0.08, meaning it needs a stricter cutoff to avoid reacting to noise that EMA naturally filters out.

Bottom line

- The only metrics worth using are wasted fraction, CV, and EMA. Max/mean cannot be tuned and Jain's index is too insensitive for this workload
- The startup assignment is not a one-time decision. How experts are placed before training begins sets the ceiling for how much every future rebalancing trigger can save
- Cooldown actively improves the quality of each trigger by filtering out marginal imbalances that would have resolved themselves
- Smoothing the signal beats lowering the threshold. EMA at $\theta=0.05$ outperforms raw wasted fraction at $\theta=0.03$ because it waits for sustained problems rather than reacting to noise at a lower bar.
- The best planner is not the most active one. the Pareto frontier shows that the configurations saving the most per trigger are precisely the ones that fire the least often

A!

Kiitos
aalto.fi